Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 – «Информатика, вычислительная техника»

Дискретная математика и математическая логика
Лабораторная работа №4
Вариант 15

Выполнил студент гр. ИВТ-24-1б:

Оглезнев Никита Михайлович
(Фамилия, Имя, Отчество)

(подпись)

Проверил:

ст. преп. каф. ИТАС
Рустамханова Г. И.
(должность, Ф.И.О руководителя от кафедры)

(оценка) (подпись)

(дата)

Пермь 2026

Содержание

Содержание 2
Цель 3
Задачи 3
Теоретический блок 4
Реализация 6
Заключение 10
Приложение А 11
Приложение Б 12

Цель

Изучение теоретических основ теории формальных языков и конечных автоматов, приобретение практических навыков построения детерминированного конечного автомата (ДКА) для распознавания заданного формального языка, разработки диаграммы Мура, таблицы переходов и регулярного выражения.

Задачи

1) Изучить определения конечного автомата, диаграммы Мура, таблицы переходов и регулярных выражений.
2) Язык L над алфавитом {a, b, c, d, e}, состоящий из слов, в которых символ 'b' встречается не раньше первого вхождения символа 'a', и количество вхождений символа 'd' строго больше 2
3) Построить диаграмму Мура для данного языка.
4) Составить таблицу переходов ДКА по диаграмме.
5) Сформулировать регулярное выражение, задающее язык L.
6) Реализовать ДКА в виде программы на языке Python.
7) Протестировать программу на различных входных строках.

Теоретический блок

1. Детерминированный конечный автомат

Детерминированный конечный автомат (ДКА) — это математическая модель вычислений, формально представляемая пятёркой $M = (Q, \Sigma, \delta, q0, F)$, где:

- Q — конечное непустое множество состояний;
- Σ — конечный входной алфавит;
- $\delta: Q \times \Sigma \to Q$ — функция переходов (детерминированная, т. е. из каждого состояния по каждому символу существует ровно один переход);
- $q0 \in Q$ — начальное состояние;
- $F \subseteq Q$ — множество принимающих (заключительных) состояний.

Автомат принимает слово $w \in \Sigma^*$, если после прочтения всех символов он оказывается в одном из состояний F. В противном случае слово отвергается.

2. Диаграмма Мура

Диаграмма Мура (диаграмма переходов) — это ориентированный граф, в котором:

- вершины соответствуют состояниям автомата;
- дуги соответствуют переходам: дуга из q в q' помечена символом a, если $\delta(q, a) = q'$;
- начальное состояние помечается входящей стрелкой без источника;
- принимающие состояния изображаются двойным кольцом.

Диаграмма наглядно отражает поведение автомата и упрощает его анализ.

3. Таблица переходов

Таблица переходов – табличное представление функции δ. Строки соответствуют состояниям, столбцы — символам алфавита. В ячейке (q, a) записывается состояние δ(q, a). Дополнительный столбец 'Вывод' указывает, является ли состояние принимающим (1) или нет (0). Ловушечное (dead) состояние явно включается в таблицу для полноты описания.

4. Регулярные выражения

Регулярное выражение (РВ) — алгебраический способ задания регулярных языков. Основные операции:

- конкатенация: xy — строка x, за которой следует y;
- альтернация (объединение): x|y — строка из x или из y;
- итерация Клини: x* — ноль или более повторений x;
- x+ = xx* (одно или более), x? = x|ε (ноль или одно вхождение).

Класс языков, задаваемых регулярными выражениями, совпадает с классом языков, распознаваемых конечными автоматами (теорема Клини).

Реализация

Алфавит: $\Sigma = \{a, b, c, d, e\}$.

Язык L состоит из всех слов $w \in \Sigma^*$, для которых одновременно выполнены два условия:

- символ 'b' может встречаться в слове только после того, как в нём уже встретился хотя бы один символ 'a'. Иными словами, ни один символ 'b' не должен предшествовать первому символу 'a'. Если символ 'a' в слове вообще отсутствует, то символ 'b' также не должен присутствовать.
- количество вхождений символа 'd' в слово строго больше 2, то есть $|w|_d > 2$ (не менее трёх вхождений 'd').

Примеры слов языка:

- «addd» $\in$ L: нет 'b' до 'a', три 'd'
- «cccadddabbb» $\in$ L: все 'b' после первого 'a', три 'd'
- «adddb» $\notin$ L: нет, слово принадлежит L — 'b' после 'a', четыре 'd'
- «bdd» $\notin$ L: 'b' до первого 'a'
- «aadd» $\notin$ L: только два 'd'

Для реализации ДКА введено 9 состояний. Состояния q0–q3 отслеживают количество вхождений d при условии, что a ещё не встречалась и b не было.

Состояния q4–q7 отслеживают количество вхождений d после появления хотя бы одного a. Состояние dead — ловушка, в которую автомат попадает при нарушении условия 1 (b до a).

Принимающие состояния: q3 (нет a, нет b, $|d|>2$) и q7 (есть a, $|d|>2$). Оба обозначены двойным кольцом на диаграмме (рис.1).

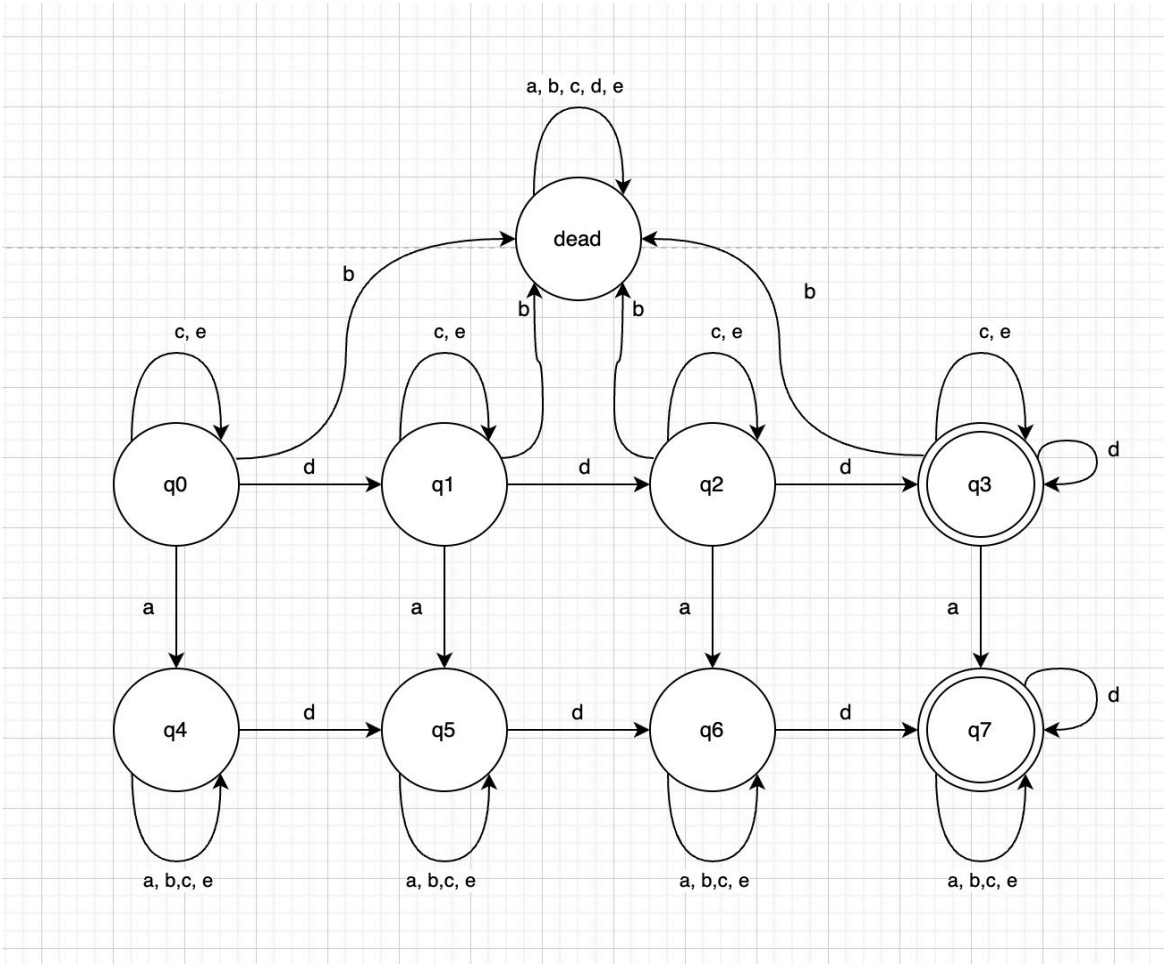


Рисунок 1 – Диаграмма Мура

Описание состояний:

- q0: начальное. 'a' не встречалась, 'b' не встречалась, |d| = 0.
- q1: 'a' не встречалась, 'b' не встречалась, |d| = 1.
- q2: 'a' не встречалась, 'b' не встречалась, |d| = 2.
- q3: 'a' не встречалась, 'b' не встречалась, |d| > 2. Принимающее.
- q4: 'a' встречалась, |d| = 0.
- q5: 'a' встречалась, |d| = 1.
- q6: 'a' встречалась, |d| = 2.
- q7: 'a' встречалась, |d| > 2. Принимающее.
- dead: ловушка. 'b' встречена до первого 'a'. Не принимающее.

Таблица переходов построена по диаграмме Мура (рис.2).

Состояние	a	b	c	d	e
q0	q4	dead	q0	q1	q0
q1	q5	dead	q1	q2	q1
q2	q6	dead	q2	q3	q2
q3	q7	dead	q3	q3	q3
q4	q4	q4	q4	q5	q4
q5	q5	q5	q5	q6	q5
q6	q6	q6	q6	q7	q6
q7	q7	q7	q7	q7	q7
dead	dead	dead	dead	dead	dead

Рисунок 2 – Таблица переходов

Получим язык L = (c|e)*d(c|e)*d(c|e)*d((c|d|e)*) | (c|e|d)*a(a|b|c|d|e)* [|d|>2].

Программа реализована на языке Python и состоит из трёх логических частей: описания алфавита и функции переходов, функции simulate (analyze), моделирующей работу ДКА на произвольном входном слове, и функции main, организующей интерактивный ввод-вывод.

Алфавит задаётся как множество Python (ALPHABET = {'a','b','c','d','e'}), что обеспечивает O(1) проверку принадлежности символа. Функция переходов delta реализована в виде вложенного словаря: внешний ключ — текущее состояние (строка), внутренний ключ — входной символ, значение — следующее состояние. Такое представление точно соответствует математическому определению δ: Q × Σ → Q и обеспечивает табличный (O(1)) поиск перехода.

Состояние dead включено явно как элемент словаря с петлями по всем символам (dead → dead), что делает автомат полным (total DFA): функция переходов определена для всех пар (состояние, символ), что исключает ошибки при обработке любых входных строк.

Функция analyze(word) принимает строку и возвращает тройку (принадлежность, конечное_состояние, количество_d). Алгоритм:

инициализируется текущее состояние START = 'q0' и счётчик вхождений 'd'. Далее в цикле для каждого символа сначала проверяется его принадлежность алфавиту (если нет — возврат ошибки), затем счётчик 'd' увеличивается при необходимости, и выполняется переход: state = delta[state][ch]. По окончании цикла возвращается флаг state in ACCEPTING, текущее состояние и счётчик.

Логика подсчёта 'd' в функции дублирует информацию, уже закодированную в состояниях автомата (q0/q4 ↔ |d|=0, q1/q5 ↔ |d|=1, q2/q6 ↔ |d|=2, q3/q7 ↔ |d|>2), и используется исключительно для информативного вывода, а не для принятия решения — решение принимается только на основе конечного состояния.

Функция main организует бесконечный цикл ввода слов. Пустая строка завершает программу. После вызова analyze результат интерпретируется по трём ветвям: слово принято; конечное состояние dead (нарушено условие 1 — 'b' до 'a'); слово отвергнуто по недостаточному числу 'd' (условие 2 не выполнено). Такая детализированная диагностика позволяет пользователю понять причину отклонения слова.

Заключение

Были решены следующие задачи: изучена теория ДКА и регулярных языков; построена диаграмма Мура с 9 состояниями (q0–q7 и dead); составлена полная таблица переходов; сформулировано регулярное выражение; реализована и протестирована программа на Python.

Практическая реализация подтверждает корректность теоретической модели: автомат принимает именно те слова, которые удовлетворяют обоим условиям варианта.

Приложение А

Результаты работы программы на тестовых файлах.

```
Вариант 15: слова, в которых 'b' встречается не раньше первого 'a'
             и количество вхождений 'd' больше 2
Алфавит: {a, b, c, d, e}
Для выхода введите пустую строку.

Слово: bad
  НЕ ПРИНАДЛЕЖИТ  |  встречено 'b' до первого 'a'  |  d встретилось: 1

Слово: baddd
  НЕ ПРИНАДЛЕЖИТ  |  встречено 'b' до первого 'a'  |  d встретилось: 3

Слово: ddd
  ПРИНАДЛЕЖИТ языку  |  d встретилось: 3  |  состояние: q3

Слово: ba
  НЕ ПРИНАДЛЕЖИТ  |  встречено 'b' до первого 'a'  |  d встретилось: 0

Слово: dddba
  НЕ ПРИНАДЛЕЖИТ  |  встречено 'b' до первого 'a'  |  d встретилось: 3

Слово: dddab
  ПРИНАДЛЕЖИТ языку  |  d встретилось: 3  |  состояние: q7

Слово: aaaaaaaddaaaaaaaaaabbbbbbbbdddddd
  ПРИНАДЛЕЖИТ языку  |  d встретилось: 7  |  состояние: q7
```

Приложение Б

Полный код программы.

```
ALPHABET = {'a', 'b', 'c', 'd', 'e'}

# Таблица переходов: delta[state][symbol] -> new_state
# dead — явная ловушка (b встречено до первого a)
delta = {
    'q0':   {'a': 'q4',   'b': 'dead', 'c': 'q0',   'd': 'q1',   'e': 'q0'  },
    'q1':   {'a': 'q5',   'b': 'dead', 'c': 'q1',   'd': 'q2',   'e': 'q1'  },
    'q2':   {'a': 'q6',   'b': 'dead', 'c': 'q2',   'd': 'q3',   'e': 'q2'  },
    'q3':   {'a': 'q7',   'b': 'dead', 'c': 'q3',   'd': 'q3',   'e': 'q3'  },
    'q4':   {'a': 'q4',   'b': 'q4',   'c': 'q4',   'd': 'q5',   'e': 'q4'  },
    'q5':   {'a': 'q5',   'b': 'q5',   'c': 'q5',   'd': 'q6',   'e': 'q5'  },
    'q6':   {'a': 'q6',   'b': 'q6',   'c': 'q6',   'd': 'q7',   'e': 'q6'  },
    'q7':   {'a': 'q7',   'b': 'q7',   'c': 'q7',   'd': 'q7',   'e': 'q7'  },
    'dead': {'a': 'dead', 'b': 'dead', 'c': 'dead', 'd': 'dead', 'e': 'dead'},
}

ACCEPTING = {'q3', 'q7'}  # q3: b не было, d>=3; q7: a было, d>=3
START = 'q0'


def analyze(word):
    state = START
    count_d = 0
    for ch in word:
        if ch not in ALPHABET:
            return False, f"недопустимый символ '{ch}'", count_d
        if ch == 'd':
            count_d += 1
        state = delta[state][ch]
    return state in ACCEPTING, state, count_d


def main():
    print("Вариант 15: слова, в которых 'b' встречается не раньше первого 'a'")
    print("              и количество вхождений 'd' больше 2")
    print("Алфавит: {a, b, c, d, e}")
    print("Для выхода введите пустую строку.\n")

    while True:
        word = input("Слово: ").strip()
        if not word:
            break

        result, state, count_d = analyze(word)

        if result:
            print(f"  ПРИНАДЛЕЖИТ языку  |  d встретилось: {count_d}  |  состояние:
{state}\n")
```

```
        else:
            if state == 'dead':
                print(f"  НЕ ПРИНАДЛЕЖИТ  |  встречено 'b' до первого 'a'  |  d
встретилось: {count_d}\n")
            elif isinstance(state, str) and state.startswith("недопустимый"):
                print(f"  ОШИБКА: {state}\n")
            else:
                print(f"  НЕ ПРИНАДЛЕЖИТ  |  d встретилось: {count_d} (нужно >2)  |
состояние: {state}\n")


if __name__ == "__main__":
    main()
```